

Full Stack Development with MERN

Project Documentation

1. Introduction

- **Project Title:** ShopEZ – Full Stack E-Commerce Web Application
- **Team Members:** Baikani Vaishnavi

Role:

- Frontend Development (React.js)
- Backend Development (Node.js & Express.js)
- Database Management (MongoDB)
- Authentication & Security (JWT)
- UI/UX Design
- Testing & Deployment

2. Project Overview

Purpose:

ShopEZ is a full-stack e-commerce web application developed to provide users with a simple, secure, and user-friendly online shopping experience. The platform allows customers to browse products, search items, filter products by category, add products to a wishlist or shopping cart, and place orders efficiently.

The primary goal of the project is to create a modern online shopping platform that improves product discovery and simplifies the purchasing process. The system also includes an admin dashboard that enables administrators to add, manage, and remove products, ensuring efficient product management. ShopEZ was developed using React.js for the frontend, Node.js and Express.js for the backend, MongoDB for database management, and JWT for secure user authentication.

Features:

- User Registration and Login with secure authentication using JWT.
- Product Browsing with an attractive and user-friendly interface.
- Product Search functionality for quickly finding products.
- Category-Based Filtering to browse products by categories such as Fashion, Beauty, Electronics, and Home Decor.
- Product Details Page displaying detailed information about each product.

- Shopping Cart Management allows users to add and remove products from their cart.
- Wishlist Feature for saving favorite products.
- Order Placement and Management for a smooth purchasing process.
- My Orders Page where users can view their previous orders.
- Featured Products Section highlighting selected products.
- Journal/Blog Section providing articles, tips, and shopping inspiration.
- Admin Dashboard for product management.
- Add Product functionality for administrators.
- Delete Product functionality for administrators.
- Responsive and Modern User Interface designed for a seamless user experience.
- MongoDB Database Integration for storing user, product, and order information.
- RESTful API Architecture for efficient frontend-backend communication.

3. Architecture

• Frontend:

The frontend of ShopEZ is developed using React.js, a component-based JavaScript library that enables the creation of interactive and dynamic user interfaces. The application is structured into reusable components and pages, making the code modular, maintainable, and scalable.

The frontend consists of various components such as Navbar, Hero Section, Product Cards, Category Cards, Journal Cards, Footer, and Feature Sections. These components are combined to create pages such as Home, Login, Register, Product Details, Cart, Wishlist, Orders, and Admin Dashboard.

React Router DOM is used for client-side routing, allowing users to navigate between different pages without reloading the application. Context API is implemented for state management, particularly for managing shopping cart items, wishlist data, and search functionality across the application.

Axios is used to communicate with the backend APIs for fetching products, managing user authentication, placing orders, and performing administrative operations. The user interface is designed with a responsive layout to ensure a smooth experience across different screen sizes and devices.

Overall, the React-based frontend provides a fast, responsive, and user-friendly shopping

experience while maintaining a clean and organized project structure.

• **Backend:**

The backend of ShopEZ is developed using Node.js and Express.js, which provide a fast, scalable, and efficient server-side environment for handling application logic and API requests.

The backend follows a structured architecture consisting of Routes, Controllers, Models, and Middleware. Express.js is used to create RESTful APIs that handle user authentication, product management, cart operations, wishlist management, and order processing.

MongoDB is used as the database, and Mongoose is utilized as the Object Data Modeling (ODM) library to define schemas and interact with the database. The application includes models such as User, Product, and Order for storing and managing data efficiently.

JWT (JSON Web Token) authentication is implemented to provide secure user login and authorization. Middleware functions are used to verify user tokens, protect private routes, and restrict access to administrative functionalities.

Controllers handle the business logic of the application, including user registration, login, product retrieval, product creation, order placement, and product management operations. Routes act as endpoints that connect client requests to the appropriate controller functions.

Axios is used on the frontend to communicate with these backend APIs, enabling seamless data exchange between the user interface and the database.

Overall, the Node.js and Express.js backend provides a secure, scalable, and organized foundation for managing users, products, orders, and administrative operations within the ShopEZ platform.

• **Database:**

ShopEZ uses MongoDB as its primary database for storing and managing application data. MongoDB is a NoSQL database that stores information in flexible JSON-like documents, making it suitable for handling dynamic e-commerce data efficiently. Mongoose is used as the Object Data Modeling (ODM) library to define schemas, validate data, and interact with the database.

Database Schema

User Collection

The User collection stores customer and administrator information. Each user document contains:

- Name
- Email Address

- Password (encrypted for security)
- Admin Status (isAdmin)
- Account Creation Details

Product Collection

The Product collection stores information about products available on the platform. Each product document contains:

- Product Title
- Price
- Product Image URL
- Category
- Stock Quantity
- Creation and Update Timestamps

Order Collection

The Order collection stores customer order information. Each order document contains:

- User Reference
- Ordered Products
- Quantity
- Total Price
- Order Status
- Order Creation Date

Database Interactions

The backend communicates with MongoDB using Mongoose models and queries. The application performs various database operations, including:

- Creating new user accounts during registration.
- Authenticating users during login.
- Retrieving product information for display on the website.
- Adding, updating, and deleting products through the Admin Dashboard.
- Managing shopping cart and wishlist data.
- Storing customer orders after checkout.
- Retrieving order history for users through the My Orders page.

MongoDB enables efficient data storage, retrieval, and management, ensuring smooth communication between the frontend and backend components of the ShopEZ platform.

4. Setup Instructions

• Prerequisites:

Before setting up and running the ShopEZ project, ensure the following software and tools are installed on the system:

Software Dependencies

- Node.js (v18 or later recommended) – JavaScript runtime environment used for backend development.
- npm (Node Package Manager) – Used for installing project dependencies and managing packages.
- MongoDB – NoSQL database used for storing user, product, and order data.
- MongoDB Compass (Optional) – Graphical interface for viewing and managing MongoDB databases.
- Visual Studio Code (VS Code) – Recommended code editor for project development.
- Git (Optional) – Version control system for managing project source code.
- Postman or Thunder Client – API testing tools used to test backend routes and endpoints.

Frontend Dependencies

- React.js
- React Router DOM
- Axios
- Lucide React Icons

Backend Dependencies

- Express.js
- Mongoose
- JSON Web Token (JWT)
- bcryptjs
- dotenv
- cors
- nodemon

System Requirements

- Windows, macOS, or Linux Operating System
- Minimum 4 GB RAM

- Stable Internet Connection for package installation

After installing the above prerequisites, the frontend and backend servers can be configured and executed to run the ShopEZ application successfully.

• **Installation:**

Step 1: Clone the Repository

Open Terminal or Command Prompt and run:

```
git clone https://github.com/your-username/shopez.git
```

```
cd shopez
```

Step 2: Install Frontend Dependencies

Navigate to the frontend directory:

```
cd client
```

```
npm install
```

This will install all required frontend packages such as React.js, React Router DOM, Axios, and Lucide React.

Step 3: Install Backend Dependencies

Open a new terminal and navigate to the backend directory:

```
cd server
```

```
npm install
```

This will install all required backend packages such as Express.js, Mongoose, JWT, bcryptjs, dotenv, cors, and nodemon.

Step 4: Configure Environment Variables

Create a file named **.env** inside the server directory and add the following variables:

```
PORT=8000
```

MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_secret_key

Replace:

- `your_mongodb_connection_string` with your MongoDB connection URL.
- `your_secret_key` with a secure secret key for JWT authentication.

Step 5: Start MongoDB

Ensure MongoDB is running locally or use MongoDB Atlas for cloud database connectivity.

Step 6: Run the Backend Server

Inside the server directory:

```
npm start
```

The backend server will start on:

`http://localhost:8000`

Step 7: Run the Frontend Application

Inside the client directory:

```
npm run dev
```

The frontend application will start on:

`http://localhost:5173`

Step 8: Access the Application

Open your browser and visit:

`http://localhost:5173`

You can now:

- Register a new account
- Login securely
- Browse products

- Search and filter products
- Add items to cart and wishlist
- Place orders
- Access the Admin Dashboard (for admin users)

Verification

After successful installation, verify that:

- MongoDB is connected successfully.
- Backend APIs are running without errors.
- Frontend loads correctly in the browser.
- User authentication works properly.
- Products are displayed from the database.

This completes the installation and setup of the ShopEZ E-Commerce Platform.

5. Folder Structure

• Client:

The frontend of ShopEZ is developed using React.js and follows a component-based architecture. The client folder is organized into reusable components, pages, context providers, and routing files to ensure maintainability and scalability.

client/

|

|— public/

| |— Static assets

|

|— src/

| |

| |— components/

| | |— Navbar.jsx

| | |— Hero.jsx

| | |— ProductCard.jsx

| | |— CategoryCard.jsx

| | |— CollectionCard.jsx

| | |— JournalCard.jsx

| | |— FeatureCard.jsx

| | |— LifestyleCard.jsx

| | |— Footer.jsx

| |

| |— pages/

| | |— Home.jsx

| | |— Login.jsx

| | |— Register.jsx

```
| | └── Cart.jsx
| | └── Wishlist.jsx
| | └── ProductDetails.jsx
| | └── Products.jsx
| | └── Orders.jsx
| | └── JournalDetails.jsx
| | └── AdminDashboard.jsx
| |
| └── context/
| | └── CartContext.jsx
| | └── WishlistContext.jsx
| | └── SearchContext.jsx
| |
| └── App.jsx
| └── main.jsx
| └── index.css
|
└── package.json
    └── vite.config.js
```

Folder Description

components/

Contains reusable UI components used throughout the application such as Navbar, Product Cards, Category Cards, Journal Cards, Footer, and other design elements.

pages/

Contains the main application pages including Home, Login, Register, Cart, Wishlist, Product Details, Orders, Journal Details, Products, and Admin Dashboard.

context/

Contains React Context API files used for global state management, including shopping cart data, wishlist data, and product search functionality.

App.jsx

Acts as the main routing component and defines all application routes using React Router DOM.

main.jsx

Application entry point responsible for rendering the React application and wrapping it with required Context Providers.

index.css

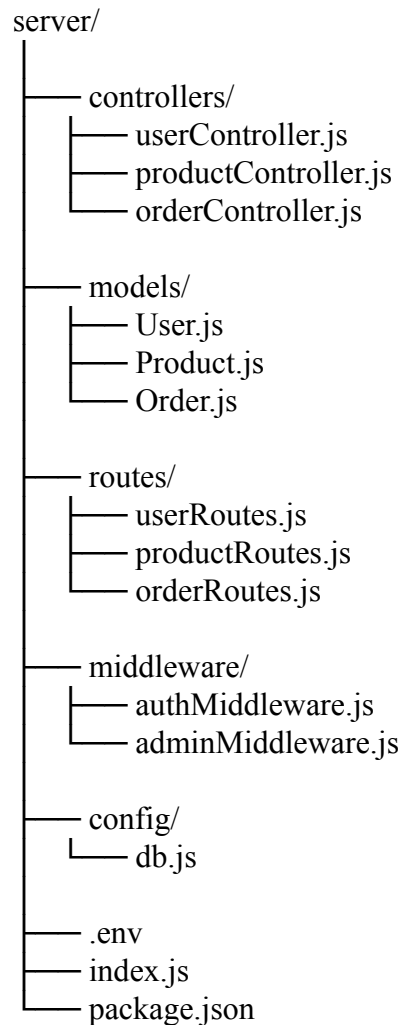
Contains global styles and common UI customization for the application.

This folder structure promotes modularity, code reusability, easier maintenance, and efficient development of the ShopEZ frontend application.

• Server:

Server (Backend) Organization

The backend of ShopEZ is developed using Node.js and Express.js and follows a modular architecture to ensure scalability, maintainability, and efficient management of application logic. The server is organized into separate folders for routes, controllers, models, middleware, and configuration files.



Folder Description

controllers/

Contains the business logic of the application. Controllers process incoming requests, interact with the database, and return appropriate responses. Examples include user authentication, product management, and order processing.

models/

Contains MongoDB schema definitions using Mongoose. These models represent the application's data structure, including Users, Products, and Orders.

routes/

Defines API endpoints and connects HTTP requests to their corresponding controller functions. Examples include authentication routes, product routes, and order routes.

middleware/

Contains reusable middleware functions such as JWT authentication, route protection, and admin authorization. These middleware functions ensure security and access control throughout the application.

config/

Stores configuration files such as MongoDB database connection setup. This helps keep configuration separate from application logic.

index.js

Acts as the main entry point of the backend server. It initializes Express, connects to MongoDB, configures middleware, and registers API routes.

.env

Stores environment variables such as database connection strings, server port numbers, and JWT secret keys. Sensitive information is kept outside the source code for security.

This organized backend structure improves code maintainability, promotes separation of concerns, and supports efficient development and future scalability of the ShopEZ platform.

6. Running the Application

After installing all dependencies and configuring the environment variables, follow the steps below to run the ShopEZ application locally.

Running the Backend Server

1. Open a terminal and navigate to the server directory:

```
cd server
```

2. Start the backend server:

```
npm start
```

or (if using nodemon):

```
npm run dev
```

3. The backend server will start on:

`http://localhost:8000`

Running the Frontend Application

1. Open another terminal and navigate to the client directory:

`cd client`

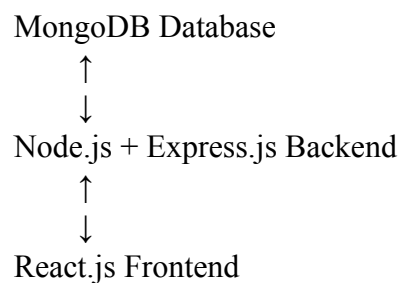
2. Start the React application:

`npm run dev`

3. The frontend application will start on:

`http://localhost:5173`

Application Workflow



Verification Steps

After starting both servers:

1. Open the browser and navigate to:

`http://localhost:5173`

2. Register a new user account.

3. Login using valid credentials.
4. Browse products and test search functionality.
5. Add products to Wishlist and Cart.
6. Place an order and verify it appears in the Orders page.
7. Login as an administrator and access the Admin Dashboard to manage products.

If all features work successfully, the ShopEZ application has been configured and executed correctly.

7. API Documentation

The ShopEZ backend exposes RESTful API endpoints for user authentication, product management, and order processing. These APIs are built using Node.js, Express.js, and MongoDB.

1. User Authentication APIs

Register User

Endpoint:

POST /api/users/register

Request Body:

```
{
  "name": "Vaishnavi",
  "email": "vaishnavi@gmail.com",
  "password": "123456"
}
```

Response:

```
{
  "_id": "12345",
  "name": "Vaishnavi",
  "email": "vaishnavi@gmail.com",
  "token": "jwt_token"
}
```

Login User

Endpoint:

POST /api/users/login

Request Body:

```
{  
  "email": "karthik@gmail.com",  
  "password": "123456"  
}
```

Response:

```
{  
  "_id": "12345",  
  "name": "Karthik",  
  "email": "karthik@gmail.com",  
  "token": "jwt_token"  
}
```

2. Product APIs

Get All Products

Endpoint:

GET /api/products

Response:

```
[  
  {  
    "_id": "1",  
    "title": "iPhone 15",  
    "price": 80000,  
    "category": "Electronics"  
  }  
]
```


Get Product by ID

Endpoint:

GET /api/products/:id

Parameters:

Parameter	Type
id	String

Response:

```
{
  "_id": "1",
  "title": "iPhone 15",
  "price": 80000,
  "image": "image_url",
  "category": "Electronics",
  "stock": 10
}
```

Add Product (Admin)

Endpoint:

POST /api/products

Headers:

Authorization: Bearer <token>

Request Body:

```
{
  "title": "Women's Summer Dress",
  "price": 2499,
  "image": "image_url",
  "category": "Fashion",
  "stock": 20
}
```

Response:

```
{
  "message": "Product created successfully"
}
```

Delete Product (Admin)

Endpoint:

DELETE /api/products/:id

Headers:

Authorization: Bearer <token>

Response:

```
{
  "message": "Product removed successfully"
}
```

3. Order APIs

Create Order

Endpoint:

POST /api/orders

Headers:

Authorization: Bearer <token>

Request Body:

```
{
  "items": [
    {
      "productId": "123",
      "quantity": 1
    }
  ],
  "totalPrice": 80000
}
```

Response:

```
{
  "message": "Order placed successfully"
}
```

Get User Orders

Endpoint:

GET /api/orders/myorders

Headers:

Authorization: Bearer <token>

Response:

```
[
  {
    "_id": "order123",
    "totalPrice": 80000,
    "createdAt": "2026-06-15"
  }
]
```

S.No	Method	Endpoint	Description
1	POST	/api/users/register	Register User
2	POST	/api/users/login	Login User
3	GET	/api/products	Get All Products
4	GET	/api/products/:id	Get Product Details
5	POST	/api/products	Add Product (Admin)
6	DELETE	/api/products/:id	Delete Product (Admin)
7	POST	/api/orders	Create Order
8	GET	/api/orders/myorders	View User Orders

8. Authentication

ShopEZ implements secure user authentication and authorization using **JSON Web Tokens (JWT)**. Authentication ensures that only valid users can access their accounts, while authorization controls access to protected resources and administrative functionalities.

User Authentication

When a user registers, their information is stored in the MongoDB database. During login, the system verifies the user's email and password. If the credentials are valid, a JWT token is generated and returned to the user.

The generated token contains user-related information and is used to identify the user during subsequent requests.

JWT Token-Based Security

After successful login, the JWT token is stored on the client side and included in the Authorization header when accessing protected API routes.

Example:

Authorization: Bearer <JWT_TOKEN>

The backend verifies the token before granting access to protected resources.

Authorization

ShopEZ uses role-based authorization to restrict access to administrative features.

Two types of users are supported:

- **Regular User:** Can browse products, manage cart and wishlist, place orders, and view personal orders.
- **Administrator:** Has all user privileges plus the ability to add and delete products through the Admin Dashboard.

Protected routes use authentication middleware to verify the JWT token and authorization middleware to verify administrative privileges.

Authentication Flow

1. User registers an account.
2. User logs in using email and password.
3. Server validates credentials.
4. JWT token is generated and returned.
5. Token is stored on the client side.

6. Token is sent with protected API requests.
7. Backend verifies the token before processing the request.
8. Authorized users gain access to protected resources.

Security Features

- JWT-based authentication.
- Protected API routes.
- Role-based admin authorization.
- Secure password storage using bcryptjs.
- Middleware-based access control.
- Prevention of unauthorized access to administrative operations.

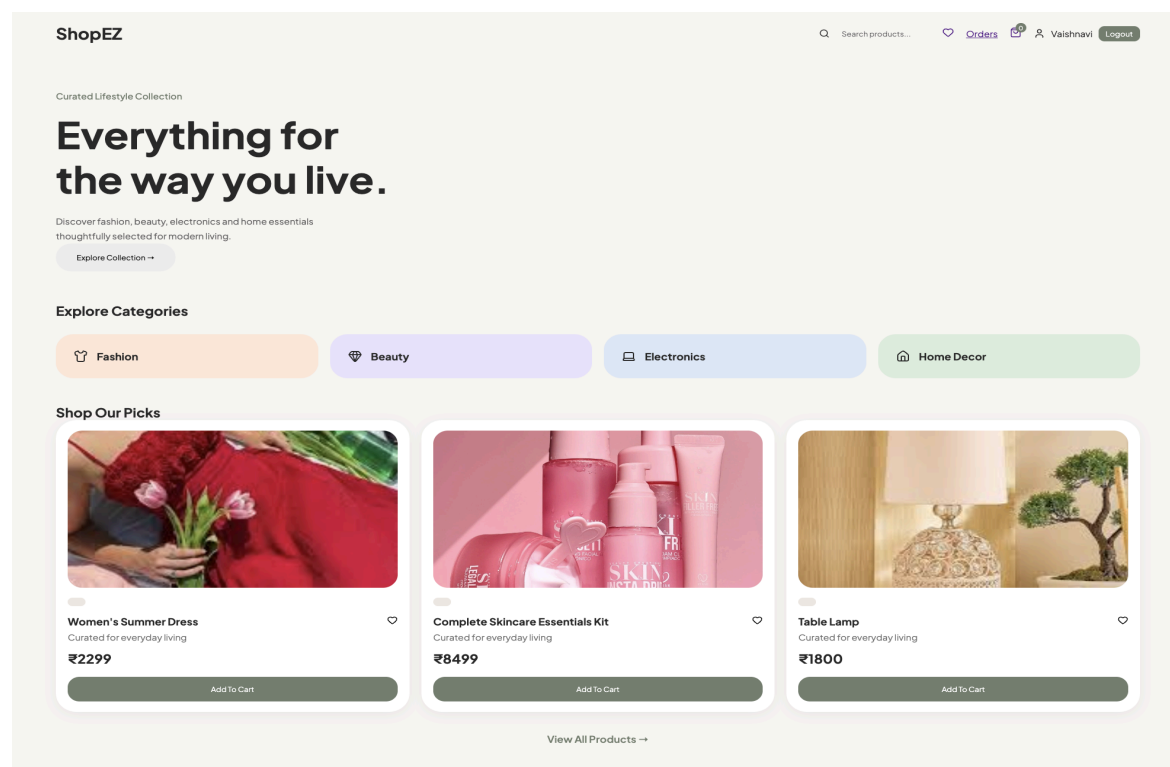
This authentication mechanism ensures secure communication between the frontend and backend while protecting sensitive user and administrative functionalities within the ShopEZ platform.

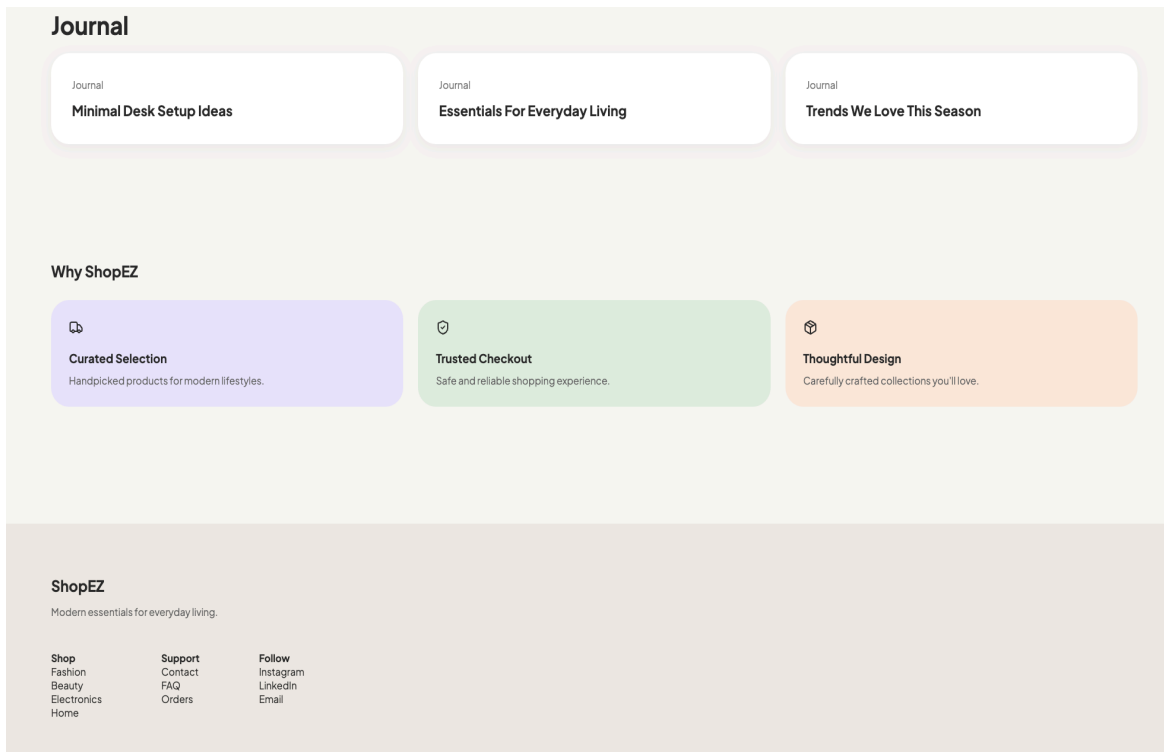
9. User Interface

The ShopEZ application provides a modern, responsive, and user-friendly interface designed to enhance the online shopping experience. The following screenshots demonstrate the major user interface components and functionalities of the system.

1.Home Page

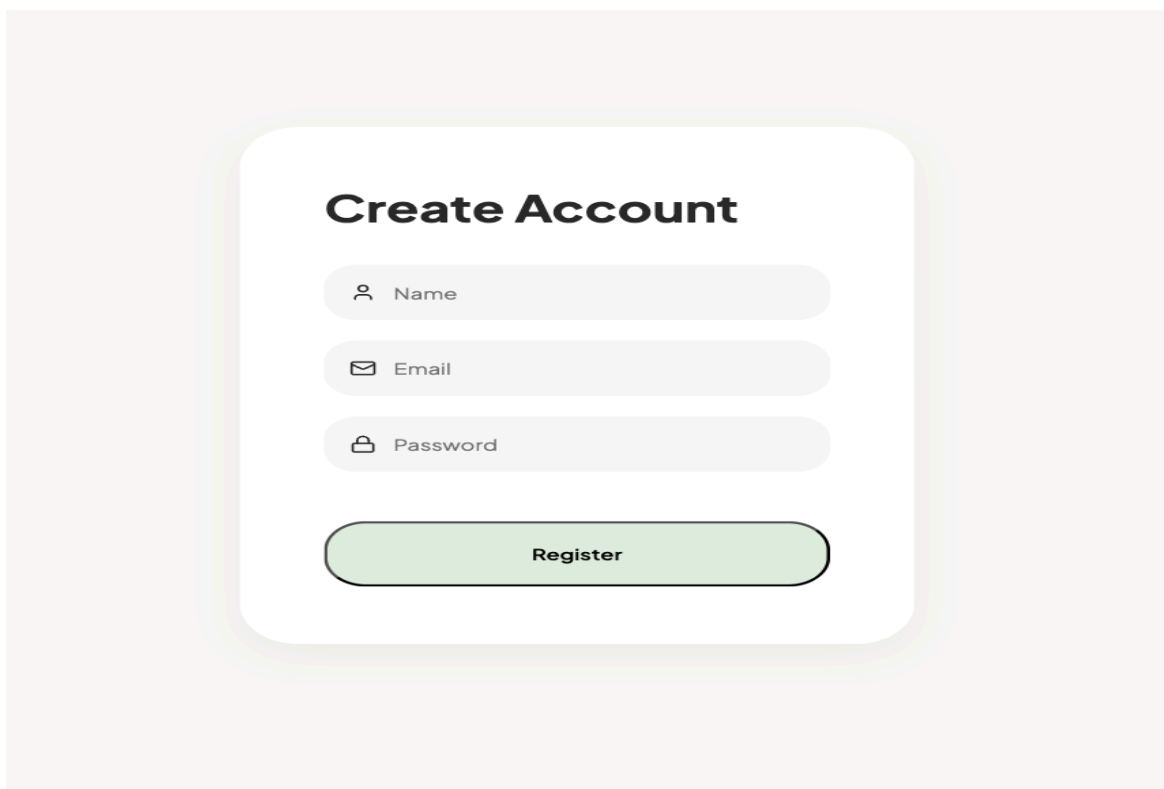
Description: Displays the hero section, product categories, featured products, journal section, and navigation menu.





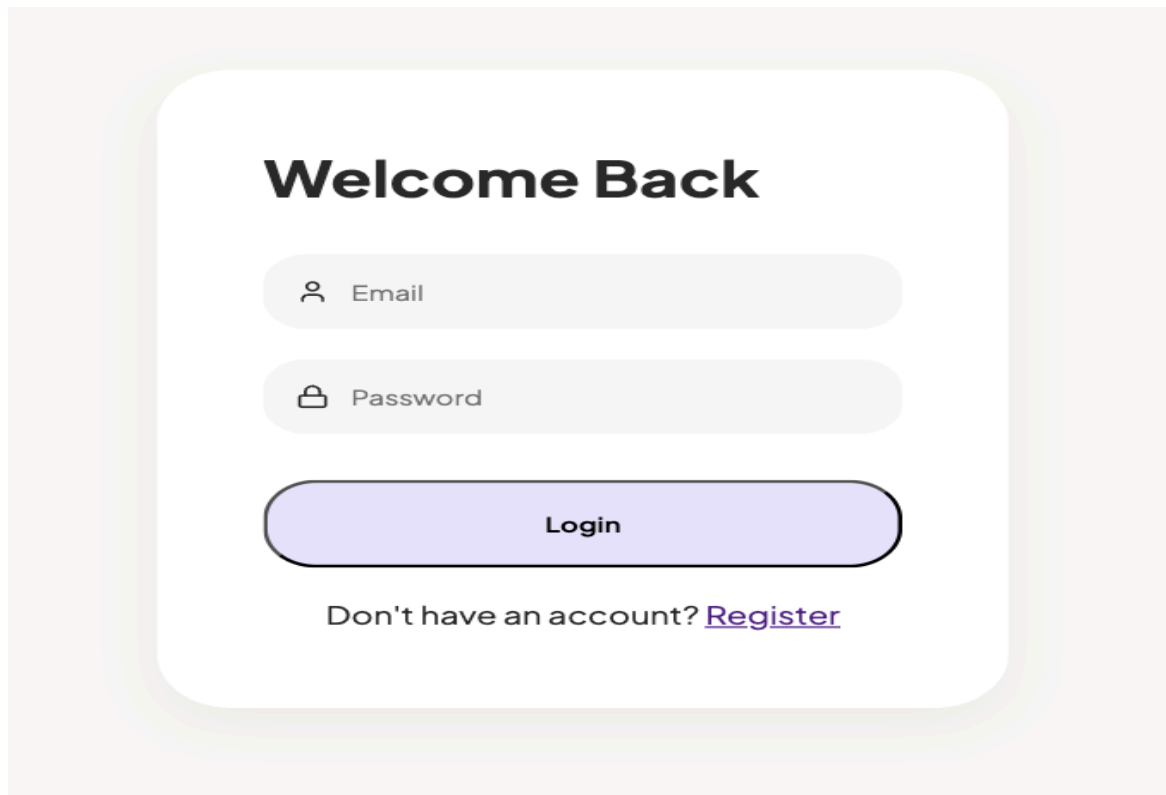
2.User Registration Page

Description: Allows new users to create an account by entering their details.



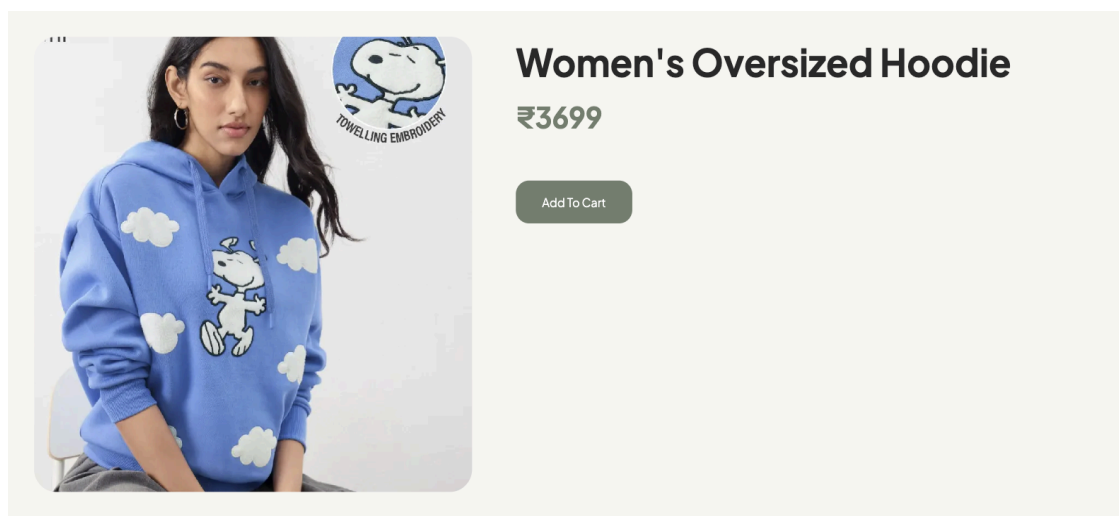
3. User Login Page

Description: Enables registered users to securely log in to the application.



4. Product Details Page

Description: Displays detailed information about a selected product, including image, price, description, and Add to Cart functionality.



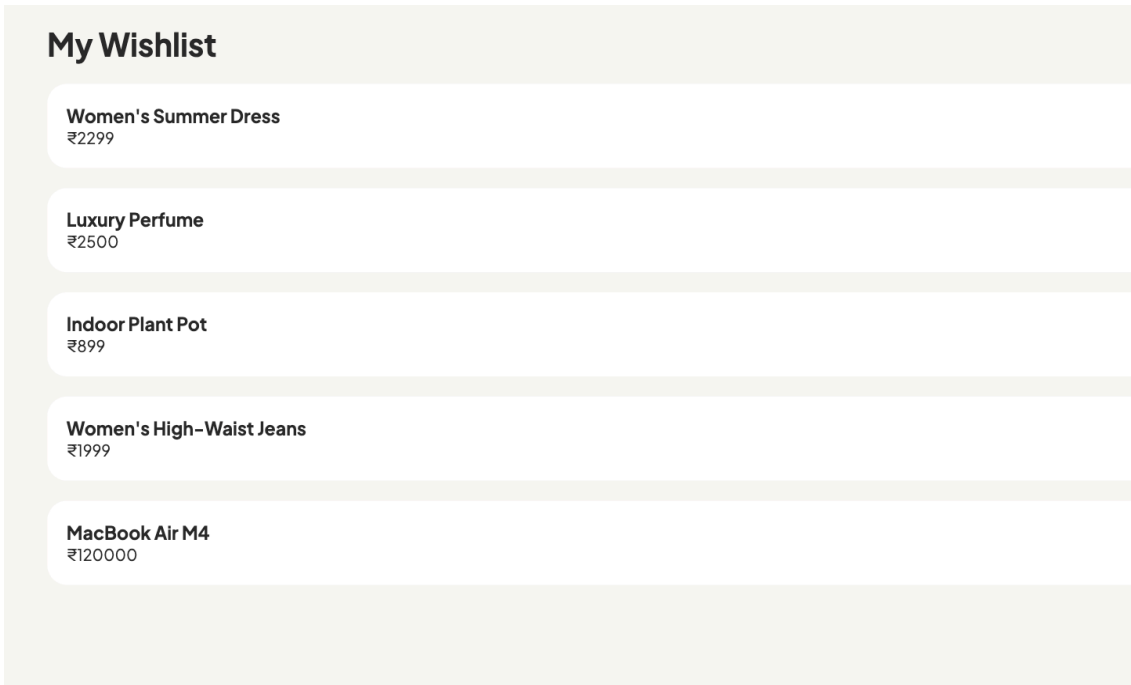
5.Shopping Cart Page

Description: Shows products added to the cart and allows users to place orders.



6.Wishlist Page

Description: Displays products saved by the user for future purchase.



7.Orders Page

Description: Shows the order history and details of orders placed by the user.

My Orders

Order #18ee63

Total Amount: ₹80000

Status: Confirmed

Order #3c9e3b

Total Amount: ₹80000

Status: Confirmed

Order #3c9e3d

Total Amount: ₹80000

Status: Confirmed

Order #4af4fa

Total Amount: ₹120000

Status: Confirmed

8.Admin Dashboard

Description: Allows administrators to add, view, and delete products from the platform.

Admin Dashboard

Title

Price

Image URL

Category

Stock

Add Product

Products

iPhone 15

₹80000

Electronics

Stock: 20

Delete

MacBook Air M4

₹120000

Electronics

Stock: 10

Delete

Smart Watch

₹12000

Electronics

Stock: 25

Delete

Gaming Headphones

₹5000

Electronics

Stock: 20

Delete

Running Shoes

₹4500

Fashion

Stock: 25

Delete

Men's White T-Shirt

₹999

Fashion

Stock: 30

Delete

10. Testing

Testing was performed throughout the development of the ShopEZ application to ensure that all functionalities operated correctly and met the project requirements. Both frontend and backend components were tested to identify and resolve errors, improve performance, and ensure a smooth user experience.

Testing Strategy

The project followed a functional testing approach where individual features were tested independently and then integrated into the complete system.

The following functionalities were tested:

- User Registration
- User Login and Authentication
- JWT Token Validation
- Product Retrieval and Display
- Product Search Functionality
- Category-Based Product Filtering
- Wishlist Management
- Shopping Cart Operations
- Product Details Page
- Order Placement
- Order History Retrieval
- Admin Product Addition
- Admin Product Deletion
- Protected Route Access
- Database Connectivity

Testing Tools Used

1. Thunder Client

Thunder Client was used within Visual Studio Code to test backend API endpoints. Various HTTP requests such as GET, POST, and DELETE were executed to verify API functionality and response accuracy.

2. Postman

Postman was used for testing REST API endpoints, validating request and response data, checking authentication tokens, and verifying protected routes.

3. Web Browser Testing

The application was tested in a web browser to verify:

- User Interface functionality
- Navigation between pages
- Form validation

- Search and filtering features
- Cart and Wishlist operations
- Order processing workflow

4. MongoDB Compass

MongoDB Compass was used to inspect database collections and verify that user, product, and order information was correctly stored and retrieved from MongoDB.

Test Results

All major functionalities of the ShopEZ platform were tested successfully. User authentication, product management, cart operations, wishlist management, order processing, and administrative features performed as expected without critical issues.

The testing process helped ensure the reliability, security, and usability of the application before deployment and demonstration.

11. Screenshots or Demo

Demo URL:

https://docs.google.com/videos/d/1qU1CHbzQD9RcNunVOeMJ9NjGWSjcusKDNvoeP1kHKkY/edit?usp=drive_link

GitHub Repository: <https://github.com/24eg105v40-droid/ShopEZ>

12. Known Issues

The ShopEZ application has been tested extensively and all major functionalities are working as expected. However, the following limitations have been identified:

- Product images are currently managed through image URLs rather than direct image uploads.
- Search functionality supports product title matching but does not include advanced filtering options.
- Category filtering is limited to predefined product categories.

- Product editing functionality is not available in the current version of the Admin Dashboard.
- Payment gateway integration is not implemented in the current release.
- Product ratings and customer reviews are not included in the current version.
- Additional mobile device testing and optimization can be performed in future versions.

Despite these limitations, all core functionalities including authentication, product management, shopping cart operations, wishlist management, and order processing are functioning successfully.

13. Future Enhancements

The ShopEZ platform has been designed with scalability and future expansion in mind. Several enhancements can be implemented to improve functionality, user experience, and overall system performance.

Potential Future Enhancements

- Product Editing functionality in the Admin Dashboard for complete product management.
- Online Payment Gateway Integration using services such as Razorpay, Stripe, or PayPal.
- Product Ratings and Reviews to allow customers to share feedback and experiences.
- Advanced Search and Filtering based on price range, ratings, categories, and product availability.
- Product Recommendation System to suggest products based on user interests and purchase history.
- Image Upload Functionality for administrators instead of manually entering image URLs.
- Order Status Tracking to provide real-time updates such as Processing, Shipped, and Delivered.
- Inventory Management System with automatic stock updates and low-stock notifications.
- Email Notifications for user registration, order confirmation, and delivery updates.
- User Profile Management for updating personal information and viewing account activity.

- Mobile Application Development using React Native for Android and iOS devices.
- Enhanced Security Features including password reset, email verification, and two-factor authentication.
- Product Comparison Feature allowing users to compare multiple products before purchasing.
- Analytics Dashboard for administrators to monitor sales, orders, and customer activity.
- Deployment on cloud platforms with automated backups and improved scalability.

These enhancements would further improve the functionality, security, and user experience of the ShopEZ platform while making it suitable for real-world commercial deployment.